```
function arcs =
ps_construct_redundant_network(psc_az,psc_r,Ncon,Nparts,max_arc_length)

% Construct a redundant network between PSC. The minimum number of arcs to
% a PSC can be set. In general, this will result in a denser network,
% compared to the result of a Delaunay triangulation.
%
% Input:    - psc_az              azimuth coordinates of the PSC
%           - psc_r               range coordinates of the PSC
%           - Ncon                minimum number of connections (arcs)
%                                 to a PSC
%           - Nparts              number of partitions of a full cycle
%                                 to which the arcs are divided
%           - max_arc_length      maximum arc length
%
% Output:   - arcs                the arcs forming the network
%
% ----------------------------------------------------------------------
% File............: ps_construct_redundant_network.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ----------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log

global az_spacing r_spacing

psc_az = psc_az*az_spacing;
psc_r = psc_r*r_spacing;

if isempty(find([2 4 6 8 10 12 16 32 64 128 256] == Nparts))
    error('The Nparts variable should be 2, 4, 6, 8, 10, 12, 16, 32, 64, 128 or
256');
end

%if Ncon<Nparts
%    Ncon = Nparts;
%    % to avoid a badly constructed network with only arcs at one side of the
PSC
%end

Npsc_temp = length(psc_az);

arcs = repmat(NaN,Npsc_temp*Ncon,2);

part_delta = 360/Nparts;
part_begin = 0:part_delta:359;
part_end = part_delta:part_delta:360;

for v = 1:Npsc_temp

    delta_az = psc_az(v)-psc_az;
    delta_r = psc_r(v)-psc_r;
    delta_r(v) = NaN;%to exclude v

    distances_vec = sqrt(delta_az.^2+delta_r.^2);
```

```
    orientations = atan2(delta_az,delta_r)*180/pi+180;

    count = 0;
    con_counter = ones(Nparts,1);
    loop_counter = 0;
    part=0;

    while (count<Ncon)&&(loop_counter<=Ncon*Nparts)
        part = mod(part,Nparts)+1;
        index = find((orientations>=part_begin(part))& ...
                     (orientations<part_end(part)));
        if ~isempty(index)
            distances = distances_vec(index);
            [distances,index2] = sort(distances);
            if
(length(index2)>=con_counter(part))&&(distances(con_counter(part))<max_arc_lengt
h)
                count = count+1;
                arcs((v-1)*Ncon+count,:) = [v index(index2(con_counter(part)))];
                con_counter(part) = con_counter(part)+1;
            end
        end
        loop_counter = loop_counter+1;
    end
end

arcs = arcs(~isnan(arcs(:,1)),:);
arcs = sort(arcs,2); % to get psc in ascending order

arcs = unique(arcs,'rows'); % remove double arcs
```